TICKET 4

Deep Gap Analysis Report

3-Score Architecture Unification

Intelligence API Layer Refactoring

Project	DeepMindQ Intelligence API
Document Type	Gap Analysis + Fixture Report
Scope	Ticket 4: 3-Score Architecture
Audit Depth	Line-by-line, cross-file
Total Findings	52 gaps (4 CRITICAL, 14 HIGH, 18 MEDIUM, 16 LOW)
Date	2026-07-31

1. Executive Summary

This report presents the findings of a line-by-line, cross-file audit of Ticket 4 (3-Score Architecture Unification) for the DeepMindQ Intelligence API Layer. The audit examined every source file related to scoring: three scoring engines, two API routes, the unified types contract, the frontend ScoreTriple component, the Prisma schema, and all existing tests. A total of 52 gaps were identified across four severity levels. Four of these are CRITICAL and must be resolved before the ticket can be considered complete.

The most significant systemic issue is that the new unified scores endpoint (GET /api/companies/{id}/scores) lives outside the Intelligence API Layer. It bypasses the established guard middleware (intelligenceGuard), meaning it has no rate limiting, no Zod validation, no correlation-id tracking, no scrubError protection, and does not use the standard IntelligenceResponse envelope. This violates the core architectural principle that all intelligence flows through the Intelligence API Layer. The tests for this endpoint are also tautological placeholder tests that validate hardcoded mock objects rather than calling the actual route handler.

A second systemic problem is the tier classification inconsistency across all three scoring systems. Intelligence Score uses lowercase tiers (hot, warm, cold), Account Priority uses uppercase (HOT, ACTIVE, NURTURE, LOW), and Revenue Opportunity uses uppercase snake_case (HOT_ACCOUNT, WARM_ACCOUNT, NURTURE, AT_RISK). The frontend ScoreTriple component renders these raw tier strings directly as badges, resulting in a jarring visual inconsistency where one gauge shows "hot" and the next shows "WARM_ACCOUNT".

Finding Summary

Severity	Count	Status
CRITICAL	4	Must fix before merge
HIGH	14	Fix in current ticket
MEDIUM	18	Fix in current or next ticket
LOW	16	Backlog / tech debt
TOTAL	52	

2. Critical Gaps (Must Fix Before Merge)

These four issues represent fundamental defects that either violate the architecture spec, expose security risks, or make the test suite meaningless. Each one must be resolved before Ticket 4 can be considered complete.

ID	Severity	File / Area	Gap Description	Fixture / Fix
C-1	CRITICAL	`scores/route.ts (L92-243)`	No rate limiting, no validation, no guard middleware on GET /api/companies/{id}/scores. The route uses bare NextRequest, does not call intelligenceGuard or utilityGuard, has no Zod schema for the companyId param, no correlation-id, no scrubError, and no response headers. This endpoint can be hammered unlimitedly and leaks the URL in error logs (line 240 logs _request.url instead of extracted id).	Rewrite GET handler to accept NextRequest. Call utilityGuard(request, "scores") at the top. Import scrubError and use it for the 500 catch. Use Response.json() instead of NextResponse.json() for consistency with all other intelligence routes. Fix logger to use extracted companyId.
C-2	CRITICAL	`ticket4-score-unification.test.ts (L1-236)`	All 14 tests in this file are tautological placeholder tests that validate hardcoded mock objects. None of them call any actual scoring function or API route handler. For example, the "Intelligence Score is always 0-100" test checks that the literal array [0, 25, 50, 75, 100] contains values in range 0-100, which is trivially true. The "scores endpoint returns correct shape" test creates a hardcoded object and checks it has the expected keys.	Replace all tests with actual integration tests that: (1) Mock db module, (2) Call GET handler from scores/route.ts with a mock Request, (3) Assert response shape from actual handler output, (4) Test edge cases: company not found, AccountScore missing, scoreBreakdown parse failure, PriorityScoreHistory empty.
C-3	CRITICAL	`account-scoring.ts (L200-212)`	classifyCategory() never returns AT_RISK. The AccountCategory type includes "AT_RISK" but this function only returns "HOT_ACCOUNT" (>=70), "WARM_ACCOUNT" (>=40), or "NURTURE" (everything below 40). The deprecated account-scorer.ts correctly handles AT_RISK (< 40), but the new scorer silently assigns NURTURE to the lowest scores. This means companies that should be flagged as AT_RISK are instead classified as NURTURE, losing critical negative-signal information.	Add AT_RISK return: if score < ACCOUNT_CATEGORY_THRESHOLDS.AT_RISK (currently -1, change to 20), return "AT_RISK". Update signal-patterns.ts: AT_RISK threshold from -1 to 20. Add tests for the AT_RISK boundary.
C-4	CRITICAL	`revenue-score route.ts (L72)`	GET /api/ai/revenue-score leaks raw error.message to the client on 500 responses. Line 72: "const message = error instanceof Error ? error.message : 'Unknown error'" then returns { error: message } with status 500. This violates the security principle established in Ticket 1 (scrubError for all error responses) and could expose database connection strings, file paths, or internal stack traces.	Import scrubError from @/lib/intelligence-api/handler. Replace raw error.message with scrubError(error.message). Also add utilityGuard for rate limiting and correlation-id.

C-1 Fixture Code: *The following shows the corrected route handler skeleton that applies the guard:*

```
import { NextRequest } from 'next/server';
import { utilityGuard, utilityCatchError, utilitySuccess } from
'@/lib/intelligence-api/guard';

export async function GET(request: NextRequest, { params }: ...) {
  const ctx = utilityGuard(request, 'scores');
  try {
    const { id } = await params;
    // ... rest of handler ...
    return utilitySuccess(ctx, responseData, 'scores');
  } catch (err) {
    return utilityCatchError(ctx, err, 502, 'INTELLIGENCE_UNAVAILABLE', 'Scores fetch
failed');
  }
}
```

3. High Severity Gaps

These 14 issues represent significant architectural violations, data integrity risks, or missing implementations that weaken the 3-Score Architecture. They should all be fixed within the current ticket.

ID	Severity	File / Area	Gap Description	Fixture / Fix
H-1	HIGH	`Cross-file: Tier system`	Three different tier classification systems use incompatible naming: Intelligence Score (hot/warm/cold/unknown), Account Priority (HOT/ACTIVE/NURTURE/LOW), Revenue Opportunity (HOT_ACCOUNT/WARM_ACCOUNT/NURTURE/AT_RISK). ScoreTriple renders these raw values as badges. The fallback tier in company/[id]/route.ts line 469 is "medium" which matches none of the AccountPriorityTier enum values (HOT/ACTIVE/NURTURE/LOW).	Create a canonical tier normalizer: normalizeTier(rawTier, scoreSystem) that maps all variants to a unified display set: { critical, high, medium, low }. Apply in ScoreTriple before rendering. Fix the "medium" fallback to "NURTURE" (a valid enum value).
H-2	HIGH	`scores/route.ts (U8)`	The scores route defines local interfaces (IntelligenceScoreDetail, AccountPriorityDetail, etc.) that duplicate but diverge from the canonical types in types.ts. The IntelligenceCompanyContext.scores in types.ts has different field names and shapes. This means the frontend must handle two incompatible response shapes depending on which endpoint it calls.	Import types from types.ts or create a shared scores-response-types.ts. Ensure both /api/companies/{id}/scores and /api/intelligence/company/{id}?include=scores use the same exported types. Delete the local interface definitions.
H-3	HIGH	`account-scorer.ts (deprecated)`	The deprecated account-scorer.ts still exists and writes a DIFFERENT scoreBreakdown format: { signalStrength, engagement, opportunityFit, timing } vs the new format: { intelligenceCoverage, signalStrength, freshness, strategicFit, engagementHistory }. If the deprecated scorer was the last to write, the scores endpoint will parse ALL ZEROS for the 5 expected keys.	Add a migration check: in scores/route.ts, detect legacy format (presence of "engagement" or "opportunityFit" keys) and return a warning field. Run a one-time migration script to recalculate all AccountScore records using the new scorer.
H-4	HIGH	`company/[id]/route.ts (L469)`	The accountPriority tier fallback is "medium" which is not a valid CompanyPriorityTier value. The enum only allows HOT, ACTIVE, NURTURE, LOW. If a company has accountPriorityScore set but priorityTier is null (race condition during first computation), the API returns an invalid tier.	Change fallback from "medium" to "NURTURE": company.priorityTier ?? "NURTURE". Add a unit test for this edge case.
H-5	HIGH	`scores/route.ts (U10)`	Revenue Opportunity breakdown parsing assumes the new 5-key format. If AccountScore.scoreBreakdown contains the legacy 4-key format from account-scorer.ts, all 5 parsed values will be 0. The route silently returns { intelligenceCoverage: 0, signalStrength: 0, ... } without indicating the data is stale.	Add format detection: check for presence of "intelligenceCoverage" key. If absent, set a "stale: true" flag in the response and attempt to map legacy keys: engagement -> engagementHistory, timing -> freshness.
H-6	HIGH	`engine.test.ts (X1-X2)`	No unit tests for scoreStaticFit(), scoreDynamicIntelligence(), or scoreTimingUrgency(). These are the three core sub-dimension scoring functions. The existing tests only exercise computeAccountPriority() at the integration level with all-zero mocks, meaning the composite score is always driven by StaticFit alone with zero Dynamic and Timing components.	Add dedicated test describe blocks for each sub-function: scoreStaticFit (test excludeIndustries, partial keyword match, null fields, empty ICP), scoreDynamicIntelligence (test null aggregates, high signal counts, zero evidence), scoreTimingUrgency (test day-0 signals, very old signals, multiple opportunities).

ID	Severity	File / Area	Gap Description	Fixture / Fix
H-7	HIGH	`ticket2-integration.test.ts (X10)`	No test for include=scores on the intelligence company endpoint. The ticket2 integration tests cover all 10 intelligence endpoints but never verify the scores section of IntelligenceCompanyContext when include=scores is requested.	Add a test case: call GET /api/intelligence/company/{id}?include=scores with mocked DB data, verify response.data.scores contains intelligence, accountPriority, and revenueOpportunity fields with correct types.
H-8	HIGH	`revenue-opportunity-engine.ts (O1)`	The revenue-opportunity-engine.ts produces a completely separate "fourth score" (opportunityScore with grade A-F) that is NOT connected to the 3-Score Architecture. Its output is stored only in AIInsight, not in AccountScore, and is not returned by either scores endpoint. It is called by POST /api/ai/revenue-score but its results are invisible to the ScoreTriple display.	Either: (a) connect revenue-opportunity-engine output to AccountScore table via upsert, (b) add it as a fourth optional field in the scores response, or (c) explicitly document it as a separate "deep scoring" feature outside the 3-Score Architecture with a deprecation plan.
H-9	HIGH	`Prisma schema (PG1)`	PriorityScoreHistory.priorityTier is String not CompanyPriorityTier enum. This allows invalid tier values like "medium" or "warm" to be stored in the history table. The Company model uses the enum correctly, but the history table bypasses type safety.	Change PriorityScoreHistory.priorityTier to CompanyPriorityTier enum. Add a migration: ALTER TABLE PriorityScoreHistory ALTER COLUMN priorityTier TYPE CompanyPriorityTier USING priorityTier::CompanyPriorityTier. Handle NULL values by defaulting to LOW.
H-10	HIGH	`Prisma schema (PG2)`	AccountScore.category is a free-form String with default "NURTURE". Valid values should be HOT_ACCOUNT, WARM_ACCOUNT, NURTURE, AT_RISK. A Prisma enum would prevent invalid values like "hot" or "WARM" (wrong case).	Create an AccountCategory enum in Prisma schema: enum AccountCategory { HOT_ACCOUNT, WARM_ACCOUNT, NURTURE, AT_RISK }. Change AccountScore.category to use the enum. Add migration.
H-11	HIGH	`company-profile-screen.tsx (F1)`	ScoreTriple is hidden on mobile (hidden lg:block at line 832). Mobile users never see the 3-score unified display. There is no mobile fallback or alternative view.	Add a mobile fallback: show a compact horizontal score bar or stacked score cards visible below lg breakpoint. Use a responsive grid: lg:grid-cols-3 grid-cols-1 with compact layout for small screens.
H-12	HIGH	`company-profile-screen.tsx (F2)`	Revenue score item maps category (e.g., "WARM_ACCOUNT") directly as tier in ScoreTriple (line 611: tier: scoresData.revenueOpportunity.category). "WARM_ACCOUNT" is a category label, not a tier name. The ScoreTriple badge displays this raw value, which looks wrong to users.	Map category to display tier before passing to ScoreTriple: const tierMap = { HOT_ACCOUNT: "High", WARM_ACCOUNT: "Medium", NURTURE: "Low", AT_RISK: "At Risk" }. Apply in the revenueScoreItem construction.
H-13	HIGH	`types.ts (T1)`	The scores type has both revenue: RevenueScore and revenueOpportunity: {...} as separate fields. RevenueScore has { score, grade, confidence, priorityTier } while revenueOpportunity has { score, category, breakdown }. Two different revenue score shapes in the same interface with no documentation explaining the relationship.	Document the distinction clearly in types.ts with JSDoc: revenue is from ScoringEngine (real-time AI scoring), revenueOpportunity is from AccountScore table (deterministic historical scoring). Consider renaming for clarity: revenueAi vs revenueDeterministic.
H-14	HIGH	`scores/route.ts (U4-U7)`	The scores route does not use scrubError(), has no correlation-id, no response headers, and uses NextResponse.json() instead of Response.json(). This makes it inconsistent with every other intelligence API route and harder to debug in production.	Adopt utilityGuard + utilitySuccess + utilityCatchError pattern. This single change fixes rate limiting, correlation-id, response headers, error scrubbing, and response format consistency.

4. Medium Severity Gaps

These 18 issues represent design inconsistencies, missing test coverage, and moderate code quality problems. They should be addressed in the current ticket or the next immediate ticket.

ID	Severity	File / Area	Gap Description	Fixture / Fix
M-1	MEDIUM	`design-system.tsx (D1)`	ScoreGauge.getColor() and scoreGaugeColor() use different color scales for the same score ranges. ScoreGauge: >=80 green, >=60 amber, >=40 yellow, <40 red. scoreGaugeColor (used by ScoreTriple): >=80 green, >=60 blue, >=40 amber, <40 red. A score of 65 renders amber in ScoreGauge but blue in ScoreTriple.	Unify the color function: delete scoreGaugeColor() and use the same getColor logic from ScoreGauge across both components. Export a shared getColor(score) function.
M-2	MEDIUM	`company/[id]/route.ts (I2, I3)`	Revenue breakdown parsing logic is duplicated between scores/route.ts (lines 193-206) and company/[id]/route.ts (lines 444-458). If one is fixed, the other must be updated too. Two different "revenue" shapes also exist: revenue (ScoringEngine output) and revenueOpportunity (AccountScore).	Extract revenue breakdown parsing into a shared utility function: parseRevenueBreakdown(scoreBreakdown: unknown). Import in both routes.
M-3	MEDIUM	`engine.ts (E1)`	getPrioritizedCompanies() uses Record for where and orderBy, defeating Prisma type safety. This could lead to runtime query errors that TypeScript cannot catch at compile time.	Use Prisma.CompanyWhereInput and Prisma.CompanyOrderByWithRelationInput types. Import from @prisma/client.
M-4	MEDIUM	`Prisma schema (PG3, PG4)`	PriorityScoreHistory has duplicate field pairs: staticFitTotal (Int) + staticFitScore (Float) store the same value. accountPriorityScore is Int while previousScore/newScore are Float. Type inconsistency for the same conceptual value.	Deprecate the Total fields (mark with @deprecated in Prisma comments). Use only the Score fields. In a future migration, remove the Total fields. Change accountPriorityScore to Float for consistency.
M-5	MEDIUM	`account-scoring.ts (S3)`	Two different "freshness" scores exist in the system: intelligence-contract.ts measures freshness by category-based half-lives (profile/signal/contact/technology), while account-scoring.ts measures it by most recent IntelligenceObject.capturedAt. Both are called "freshness" but measure fundamentally different things.	Rename one: intelligence-contract.ts keeps "freshnessScore", account-scoring.ts renames to "dataRecency" or "signalRecency". Update the scoreBreakdown keys accordingly.
M-6	MEDIUM	`engine.test.ts (X3-X5)`	Missing tests for: scoreTimingUrgency edge cases (day-0, very old signals), getPrioritizedCompanies with sortBy/search/tier filter, computeAllAccountPriorities with partial failures (Promise.allSettled rejection handling).	Add: timing edge case tests (latestSignalDaysAgo=0, =365), sort/filter tests for getPrioritizedCompanies, rejection handling test for batch compute.
M-7	MEDIUM	`scores/route.ts (U9)`	getAccountIntelligence() failure degrades silently. If the research engine throws, the response falls back to stored Company.intelligenceScore without any indication that the score is stale. The caller cannot distinguish between a fresh live-computed score and a days-old stored fallback.	Add staleness indicator: include a "freshness" field in the intelligence response. If fallback was used, set freshness.status = "stale" with a lastComputedAt timestamp from Company.lastEnrichedAt.
M-8	MEDIUM	`account-scoring.ts (S1)`	computeStrategicFit() uses hardcoded industry keyword lists (TECH_INDUSTRY_KEYWORDS, FINANCE_INDUSTRY_KEYWORDS) that are not configurable via the ICP profile. This means the "strategic fit" dimension is vendor-defined rather than customer-defined, contradicting the ICP-driven architecture.	Accept ICP profile as parameter to computeStrategicFit(). Use ICP.targetIndustries for primary matching, fall back to hardcoded lists only when ICP is empty.

ID	Severity	File / Area	Gap Description	Fixture / Fix
M-9	MEDIUM	account-scorer.ts (R3)	Deprecated calculateAndPersistScore() uses db.accountScore.create() (not upsert). Calling it twice for the same company throws a unique constraint violation. The new account-scoring.ts correctly uses upsert.	No fix needed (file is deprecated), but add a comment warning: "Will throw on duplicate companyId. Use account-scoring.ts persistAccountScore() instead."
M-10	MEDIUM	types.ts (T1, CI-5)	IntelligenceCompanyContext.scores.revenue is typed as RevenueScore (from scoring-engine) while scores.revenueOpportunity has a custom inline type. The relationship between these two fields is undocumented.	Add JSDoc comments: "@field revenue - Real-time AI revenue score from ScoringEngine. @field revenueOpportunity - Deterministic revenue score from AccountScore table."
M-11	MEDIUM	Prisma schema (PG5)	No @@index([triggerType]) on PriorityScoreHistory. Filtering by trigger type in analytics queries would require a full table scan.	Add @@index([triggerType]) to PriorityScoreHistory model.
M-12	MEDIUM	company-profile-screen.tsx (F4)	ScoreGauge segments (line 616-620) use synthetic approximations: (score * 0.4) + 20. These are NOT the actual sub-dimension scores from the intelligence contract. They are fake derived values that mislead users.	Fetch actual breakdown from scoresData.intelligence.breakdown (6 components) when available. Use real values for segments. Fall back to synthetic only when breakdown is null.
M-13	MEDIUM	ARCHITECTURE.md (A1)	Ticket 4 spec references "account-scorer.ts" for Opportunity Score unification, but the actual implementation uses "account-scoring.ts" (the new 5-dimension scorer). The spec is stale and references the deprecated file.	Update ARCHITECTURE.md to reference account-scoring.ts. Add note about deprecated account-scorer.ts.
M-14	MEDIUM	types.ts (CI-6)	IntelligenceCompanyContext.company.intelligenceScore is number (not nullable) while accountPriorityScore is number | null. This is correct per Prisma defaults but the inconsistency is undocumented.	Add JSDoc: "intelligenceScore defaults to 0 (never null), accountPriorityScore is null until first computation."
M-15	MEDIUM	ticket2-integration.test.ts (X11)	Tests mock db.accountScore.findUnique but do not verify the revenueOpportunity sub-shape is correctly constructed from the AccountScore record in the include=scores path.	Add assertion: expect(response.data.scores.revenueOpportunity.breakdown.intelligenceCoverage).toBeDefined()
M-16	MEDIUM	intelligence-contract.ts (C1)	AccountIntelligence.tier uses hot/warm/cold/unknown while AccountPriorityBreakdown.tier uses HOT/ACTIVE/NURTURE/LOW. Case-sensitive tier names mixed across the system.	See H-1 (tier normalization). Part of the same systemic issue.
M-17	MEDIUM	account-scoring.ts (S4)	recalculateAllScores() is sequential (one persistAccountScore per company in a for loop). For large datasets (1000+ companies), this will be very slow.	Batch in groups of 10 using Promise.all with concurrency limit. Add progress logging.
M-18	MEDIUM	engine.ts (E5-E6)	parseEmployeeRange() is untested (edge cases: "10K+", "1,000-5,000", empty string). Tech stack JSON parse errors are silently swallowed without logging.	Add parseEmployeeRange tests. Add logger.warn for tech stack parse failures.

5. Low Severity Gaps (Backlog)

These 16 items are minor code quality improvements, documentation gaps, and minor accessibility issues. They can be deferred to a tech-debt backlog but should not be forgotten.

ID	Severity	File / Area	Gap Description	Fixture / Fix
L-1	LOW	design-system.tsx (D3)	No aria-label on ScoreTriple component for screen readers.	Add aria-label="Score summary" to the outer div.
L-2	LOW	company-profile-screen.tsx (F3)	Intelligence score fallback (line 589) always constructs a ScoreItem even when score is 0. A company with no intelligence shows "0 - unknown" instead of nothing.	Return null when score is 0 and no live computation exists.
L-3	LOW	Prisma schema (PG6)	AccountScore has no updatedAt field. No way to distinguish creation vs modification time.	Add updatedAt DateTime @updatedAt to AccountScore.
L-4	LOW	Prisma schema (PG7)	No composite index [companyId, priorityTier] on Company for tier-filtered queries.	Add @@index([companyId, priorityTier]) to Company model.
L-5	LOW	Prisma schema (S6)	PriorityScoreHistory.priorityTier is String not CompanyPriorityTier enum.	Covered by H-9. Duplicate.
L-6	LOW	intelligence-contract.ts (C2)	getAccountIntelligence() name is overloaded: computes intelligence SCORE, not general intelligence.	Consider renaming to computeIntelligenceScore(). Low priority.
L-7	LOW	scores/route.ts (U11)	Line 240 logs companyId as _request.url (full URL) instead of extracted id.	Fix to: companyId: id.
L-8	LOW	company/[id]/route.ts (I4)	Line 463: const intelScore = (company.intelligenceScore as number) ?? 0. The "as number" cast is misleading since the field is already number | null.	Change to: (company.intelligenceScore ?? 0).
L-9	LOW	engine.test.ts (X6)	No test for computeAllAccountPriorities() when some companies fail (Promise.allSettled rejection handling).	Add test with one company throwing, verify others still process.
L-10	LOW	ARCHITECTURE.md (A3)	Ticket 4 exit criteria checkboxes show [ ] (unchecked) but PROJECT_STATUS.md says COMPLETE.	Update ARCHITECTURE.md checkboxes to [x].
L-11	LOW	PROJECT_STATUS.md (P1)	Test count "1453/1453" but only 28 new tests for a 3-score unification seems low.	After fixing tests (C-2), update count.
L-12	LOW	design-system.tsx	ScoreTriple strokeDashoffset calculation does not clamp to 0. If score > 100, offset goes negative causing rendering artifacts.	Add Math.max(0, ...) to the offset calculation.
L-13	LOW	account-scoring.ts (S2)	computeStrategicFit() is not configurable via ICP profile. (Covered by M-8.)	See M-8.
L-14	LOW	engine.ts (E3)	parseEmployeeRange() returns max: Infinity for "10000+". Would serialize to null in JSON, but no JSON path exists.	No fix needed currently.
L-15	LOW	revenue-opportunity-engine.ts (O2)	scoreRevenueOpportunity() calls createInsight() which has side effects. Scoring is not idempotent.	Add idempotency key or check for existing insight before creating.
L-16	LOW	All	No concurrent score computation tests. Race condition on Company.update is possible.	Add test: two simultaneous computeAccountPriority calls for same company.

6. Test Coverage Fixtures

The following test fixture code addresses the most critical test gaps identified above. These fixtures should be added to the existing test files to provide meaningful coverage for the scoring functions.

6.1 Scores Route Integration Test Fixture

This fixture replaces the tautological tests in ticket4-score-unification.test.ts with actual route handler tests that mock the database and call the real GET handler:

```
// ticket4-score-unification.test.ts - REPLACEMENT FIXTURE
import { describe, it, expect, vi, beforeEach } from 'vitest'
import { GET } from '@/app/api/companies/[id]/scores/route'

const mockDbFindUnique = vi.fn()
const mockDbFindMany = vi.fn()
vi.mock('@/lib/db', () => ({
  db: {
    company: { findUnique: (...a: unknown[]) => mockDbFindUnique(...a) },
    accountScore: { findUnique: (...a: unknown[]) => mockDbFindUnique(...a) },
    priorityScoreHistory: { findMany: (...a: unknown[]) => mockDbFindMany(...a) },
  }
}))

vi.mock('@/lib/intelligence-contract', () => ({
  getAccountIntelligence: vi.fn().mockResolvedValue({
    intelligenceScore: 72, tier: 'hot', computedAt: '2026-01-01',
    components: { dataCompleteness: 80, evidenceQuality: 70 }
  })
}))

describe('GET /api/companies/{id}/scores', () => {
  it('returns 404 for non-existent company', async () => {
    mockDbFindUnique.mockResolvedValueOnce(null)
    const res = await GET(new Request('http://x/api/companies/nope/scores'),
      { params: Promise.resolve({ id: 'nope' }) })
    expect(res.status).toBe(404)
  })
  it('returns all 3 scores with correct shape', async () => {
    mockDbFindUnique.mockResolvedValueOnce({
      id: 'c1', rawName: 'Test', intelligenceScore: 72,
      accountPriorityScore: 85, priorityTier: 'ACTIVE',
      priorityComputedAt: new Date(), lastEnrichedAt: new Date()
    })
    mockDbFindUnique.mockResolvedValueOnce({
      companyId: 'c1', score: 68, category: 'WARM_ACCOUNT',
      scoreBreakdown: JSON.stringify({
        intelligenceCoverage: 70, signalStrength: 65,
        freshness: 80, strategicFit: 55, engagementHistory: 40
      })
    })
    mockDbFindMany.mockResolvedValue([])
    const res = await GET(new Request('http://x/api/companies/c1/scores'),
      { params: Promise.resolve({ id: 'c1' }) })
    const data = await res.json()
    expect(data.intelligence.score).toBe(72)
    expect(data.accountPriority.score).toBe(85)
    expect(data.revenueOpportunity.score).toBe(68)
    expect(data.revenueOpportunity.breakdown.intelligenceCoverage).toBe(70)
```

```
    expect(data.history).toEqual([])
  })
})
```

6.2 Engine Sub-Function Test Fixture

These tests cover the untested sub-dimension scoring functions in engine.ts:

```
// engine.test.ts - ADDITIONAL SUB-FUNCTION TESTS
describe('scoreStaticFit edge cases', () => {
  it('returns 0 industry when company is in excludeIndustries', async () => {
    mockSystemSettingFindUnique.mockResolvedValueOnce({
      value: JSON.stringify({ targetIndustries: ['SaaS'],
      excludeIndustries: ['Healthcare'] })
    })
    mockDbFindUnique.mockResolvedValueOnce({
      id: 'c1', industry: 'Healthcare', sizeRange: null, country: null,
      accountPriorityScore: null, priorityTier: null
    })
    const result = await computeAccountPriority('c1')
    expect(result.priority.staticFit.industry).toBe(0)
  })
  it('handles partial keyword overlap (20 points)', async () => {
    mockSystemSettingFindUnique.mockResolvedValueOnce({
      value: JSON.stringify({ targetIndustries: ['Financial Services'] })
    })
    mockDbFindUnique.mockResolvedValueOnce({
      id: 'c2', industry: 'Finance', sizeRange: null, country: null,
      accountPriorityScore: null, priorityTier: null
    })
    const result = await computeAccountPriority('c2')
    expect(result.priority.staticFit.industry).toBe(20)
  })
  it('returns neutral score (15) when ICP has no target industries', async () => {
    mockSystemSettingFindUnique.mockResolvedValueOnce({
      value: JSON.stringify({ targetIndustries: [] })
    })
    mockDbFindUnique.mockResolvedValueOnce({
      id: 'c3', industry: 'Unknown', sizeRange: null, country: null,
      accountPriorityScore: null, priorityTier: null
    })
    const result = await computeAccountPriority('c3')
    expect(result.priority.staticFit.industry).toBe(15)
  })
})

describe('scoreTimingUrgency edge cases', () => {
  it('returns max signalRecency (40) for same-day signals', async () => {
    mockCompanySignalFindMany.mockResolvedValueOnce([{
      impact: 'high', signalDate: new Date().toISOString(), createdAt: new Date()
    }])
    const result = await computeAccountPriority('c1')
    expect(result.priority.timingUrgency.signalRecency).toBe(40)
  })
  it('returns min signalRecency (3) for very old signals (365+ days)', async () => {
    const oldDate = new Date(Date.now() - 400 * 86400000)
    mockCompanySignalFindMany.mockResolvedValueOnce([{
      impact: 'low', signalDate: oldDate.toISOString(), createdAt: oldDate
    }])
    const result = await computeAccountPriority('c1')
```

```
    expect(result.priority.timingUrgency.signalRecency).toBe(3)
  })
})
```

7. Remediation Priority Plan

The following table outlines the recommended order for fixing all identified gaps. Items are grouped into three phases: Phase 1 (must complete before Ticket 4 can close), Phase 2 (should complete in the current sprint), and Phase 3 (backlog for future tickets).

Phase	Gap IDs	Action	Est. Effort
1 - Must	C-1, H-14	Rewrite scores/route.ts with utilityGuard + scrubError + Response.json	2h
1 - Must	C-2, H-6	Replace tautological tests with actual route + engine integration tests	3h
1 - Must	C-3, C-4	Fix AT_RISK classification + scrubError in revenue-score route	1h
1 - Must	H-1, H-4, H-12	Tier normalization + fix "medium" fallback + category-to-tier mapping	2h
2 - Should	H-2, H-5, M-2	Unify response types + extract shared breakdown parser + detect legacy format	2h
2 - Should	H-3, M-13	Add legacy format detection + migration script + update ARCHITECTURE.md	1.5h
2 - Should	H-9, H-10	Add Prisma enums for PriorityScoreHistory.priorityTier and AccountScore.category	1h
2 - Should	H-7, M-6, M-15	Add include=scores integration tests + edge case tests	1.5h
2 - Should	H-11, H-12	Add mobile ScoreTriple fallback + fix category display mapping	1.5h
3 - Later	M-4, M-5, M-8	Schema cleanup (deprecate Total fields, rename freshness, ICP-driven strategicFit)	3h
3 - Later	H-8, H-13	Decide revenue-opportunity-engine integration + document revenue types	2h
3 - Later	M-1, L-1 through L-16	All LOW + remaining MEDIUM gaps (color unification, aria labels, indexes)	4h

Total Estimated Effort: 24.5 hours across 3 phases

Phase 1 (8 hours) is the minimum required to close Ticket 4. Phases 2 and 3 should be scheduled in the following sprint cycles but are not blockers for the Ticket 4 completion gate.

8. Cross-File Reference Map

This map shows how the three scoring systems connect (or fail to connect) across the codebase. Understanding these data flows is essential for fixing the architectural gaps identified above.

Score System	Engine	Storage	API Endpoint	Frontend
Intelligence	intelligence-contract.ts	Company.intelligenceScore	/api/intelligence/company/{id}?include=scores	ScoreTriple + ScoreGauge

Score System	Engine	Storage	API Endpoint	Frontend
Account Priority	`engine.ts`	`Company.accountPriorityScore + PriorityScoreHistory`	`/api/companies/{id}/scores`	`ScoreTriple`
Revenue (Active)	`account-scoring.ts`	`AccountScore table`	`/api/companies/{id}/scores`	`ScoreTriple`
Revenue (Legacy)	`account-scorer.ts (DEPRECATED)`	`AccountScore table (WRONG FORMAT)`	`None (orphaned)`	`None`
Revenue (AI Deep)	`revenue-opportunity-engine.ts`	`AIInsight only`	`/api/ai/revenue-score`	`None (not in ScoreTriple)`